SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	S.N.V.S Karthik	**Reg. No.:**	192424186
Section / Batch:	AI&DS	**Date:**	

Code of Conduct

I _____S.N.V.S Karthik (192424186)_____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____**karthik**_____

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."
Agent: "Sure, what problem are you facing?"
User: "I do not understand machine learning."
Agent: "I can explain the basic concepts to you."
Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking
Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.
Apply your algorithm to:
User: "I want to book a flight."
Agent: "Where would you like to travel?"
User: "I want to go to Delhi."
Agent: "From which city?"
User: "From Chennai."
Your algorithm should maintain the following slots:
Departure City
Destination City
Travel Date
Number of Passengers
Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

ASSESSMENT TOOL 2 (PSEUDOCODE WRITING TASK)

Answer 1: Reference Resolution Using Multiple Constraints

Problem Understanding

Input: A discourse containing pronouns (she, her, it) and candidate noun-phrase antecedents (Meena, Kavya, a laptop). Output: The discourse with every pronoun correctly resolved to its antecedent. Constraints to apply: gender agreement, number agreement, recency of mention, and semantic/selectional compatibility with the surrounding verb.

Algorithm (Pseudocode)

```
ALGORITHM ResolveReference(Discourse D)
INPUT  : Discourse D = sequence of sentences
OUTPUT : Discourse with pronouns replaced by resolved antecedents

1.  BEGIN
2.     entities  <- ExtractEntities(D)              // NER + noun-phrase chunking
3.     pronouns  <- ExtractPronouns(D)
4.     FOR EACH pronoun p IN pronouns DO
5.        candidates <- GetPrecedingEntities(p, D)   // all NPs occurring before p
6.        candidates <- FilterByGender(candidates, Gender(p))
7.        candidates <- FilterByNumber(candidates, Number(p))
8.        IF candidates = EMPTY THEN
9.           REPORT "No antecedent found for" p ; CONTINUE
10.       scoreTable <- {}
11.       FOR EACH c IN candidates DO
12.          recencyScore   <- 1 / (SentenceDistance(c,p) + 1)
13.          semanticScore  <- SemanticCompatibility(c, VerbFrame(p))
14.          salienceScore  <- SyntacticRoleWeight(c)       // subject > object > oblique
15.          scoreTable[c]  <- w1*recencyScore + w2*semanticScore + w3*salienceScore
16.       antecedent <- ARGMAX(scoreTable)
17.       Replace(p, antecedent, D)
18.    RETURN D
```

19. END

Application to the Given Discourse

Discourse: "Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Step 1 – Entities identified: Meena (person, F, sing.), Kavya (person, F, sing.), a laptop (object).

Step 2 – Referring expressions: she(1), one, her(1), She(2), her(2), it.

Step 3 – Candidate generation and constraint filtering:

* she(1): candidates {Meena, Kavya} – both female/singular, so gender/number cannot decide alone. Recency favours Kavya (nearer NP). Semantic compatibility: the verb frame "needed X for project" fits the receiver of the laptop -> Kavya. Resolved: she(1) = Kavya.
* one: indefinite type-anaphora -> refers to "a laptop" (a new laptop, not the same physical object).
* her(1) (in "her project"): coreferent with the subject just resolved -> Kavya.
* She(2): candidates {Meena, Kavya}; verb "thanked" implies the gift-receiver thanks the giver, so the agent of "thank" = Kavya (semantic/thematic-role compatibility). Resolved: She(2) = Kavya.
* her(2) (object of "thanked"): a pronoun cannot corefer with the subject of the same clause (disjoint-reference/binding constraint) -> excludes Kavya -> resolves to Meena.
* it: non-human antecedent required -> the laptop.

Step 4 – Most appropriate antecedents selected as above using the combined weighted score.

Step 5 – Resolved discourse: "Meena gave Kavya a laptop because Kavya needed a laptop for Kavya's project. Kavya thanked Meena and started using the laptop."

Explanation

When two candidates (Meena and Kavya) share identical gender and number, hard grammatical constraints alone cannot disambiguate them. The algorithm therefore falls back on weighted soft constraints — recency (closer mentions are preferred), syntactic salience (subjects are more salient than objects), semantic/selectional compatibility (matching the pronoun to the thematic role required by the verb, e.g., "needed" implies a recipient, "thanked" implies the gift-receiver as agent), and binding constraints that forbid a pronoun from co-referring with a co-argument of the same clause. Combining these weighted scores instead of relying on a single feature prevents the algorithm from mis-resolving pronouns whenever multiple antecedents share the same grammatical properties.

Answer 2: Algorithm for Entity Chains and Discourse Coherence

Problem Understanding

Input: A short discourse containing an entity that is repeatedly referred to using full noun phrases, pronouns, and synonyms. Output: The set of entity chains present in the text and a coherence judgement based on how continuously each entity is referred to across sentences.

Algorithm (Pseudocode)

```
ALGORITHM BuildEntityChains(Discourse D)
1.  sentences <- Split(D)
2.  mentions  <- []
3.  FOR EACH sentence s IN sentences DO
4.      nps <- ExtractNounPhrasesAndPronouns(s)
5.      mentions.append((nps, index(s)))
6.  chains <- []
7.  FOR EACH mention m IN mentions (in textual order) DO
8.      bestChain <- NULL ; bestScore <- 0
9.      FOR EACH chain C IN chains DO
10.         score <- CorefScore(m, LastMention(C))   // string match, synonym match
                                                     // (e.g., laptop~computer), pronoun
                                                     // agreement, sentence distance
11.         IF score > threshold AND score > bestScore THEN
12.             bestChain <- C ; bestScore <- score
13.     IF bestChain != NULL THEN Add(m, bestChain)
14.     ELSE  chains.append(NewChain(m))
15. FOR EACH chain E IN chains DO
16.     continuity[E] <- Count(SentencesContaining(E)) / TotalSentences(D)
17. coherenceScore <- WeightedAverage(continuity, chainLength)
18. IF coherenceScore >= threshold THEN verdict <- "Coherent"
19. ELSE verdict <- "Incoherent"
20. RETURN chains, coherenceScore, verdict
```

Application to the Given Text

Text: "Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Step 1 – Entities: Anjali (person); laptop/computer (object).

Step 2 – Linking expressions of the same entity:

Chain A (person): Anjali (S1) -> She (S3) -> Anjali (S4)

Chain B (object): a new laptop (S1) -> The laptop (S2) -> it (S3) -> the computer (S4, synonym link)

Step 3 – Entity chains constructed as above (both chains span the entire 4-sentence discourse).

Step 4 – Continuity: Chain A appears in 3/4 sentences = 0.75; Chain B appears in 4/4 sentences = 1.0.

Step 5 – Coherence score = weighted average ~ 0.875, which exceeds the coherence threshold, so the discourse is judged COHERENT — every sentence connects to at least one earlier sentence through a shared entity, with no abrupt topic gaps.

Explanation

If an entity chain is broken — i.e., an entity introduced in one sentence is never referred to again while a new, unrelated entity is introduced abruptly — the discourse loses its shared "center" between adjacent sentences (a rough-shift in Centering-Theory terms). This forces the reader to reconstruct connections through inference rather than direct reference, increases referential ambiguity, and lowers the computed continuity score for that chain, which in turn reduces the overall coherence score returned by the algorithm.

Answer 3: Algorithm for Identifying Discourse Relations

Problem Understanding

Input: A discourse consisting of several sentences describing an event sequence. Output: The discourse relation (Cause-Effect, Sequence, Problem-Solution, Elaboration) that holds between each pair of consecutive discourse units, using explicit connective markers where present and contextual/semantic inference otherwise.

Algorithm (Pseudocode)

```
ALGORITHM IdentifyDiscourseRelations(D)
1. EDUs <- SegmentIntoUnits(D)              // Elementary Discourse Units (clause/sentence level)
2. markerLexicon <- {
      "as a result","therefore","because" -> Cause-Effect ;
      "after that","then","next"          -> Sequence ;
      "however","but"                     -> Contrast ;
      "for example","in addition"         -> Elaboration }
3. relations <- []
4. FOR i = 2 TO Length(EDUs) DO
```

```
5.  u1 <- EDUs[i-1] ; u2 <- EDUs[i]
6.  marker <- DetectExplicitMarker(u2, markerLexicon)
7.  IF marker != NULL THEN
8.      relation <- markerLexicon[marker]
9.  ELSE
10.     relation <- InferImplicitRelation(u1, u2)   // tense/aspect + world knowledge
11.  IF ContainsProblemCue(u1) AND ContainsFixCue(u2) THEN
12.     relation <- Problem-Solution                // contextual override
13.  relations.append( (u1, u2, relation) )
14. discourseTree <- BuildRSTTree(EDUs, relations)      // nucleus-satellite hierarchy
15. RETURN discourseTree
```

Application to the Given Discourse

Text: "The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Step 1 – Discourse units: U1 = "The server crashed unexpectedly." | U2 = "…users could not access the application." | U3 = "The technical team restarted the server." | U4 = "…the system became available again."

Step 2 – Explicit markers found: "As a result" before U2; "After that" before U4.

Step 3–4 – Relation assignment:

(U1, U2): marker "As a result" -> Cause-Effect (crash causes inaccessibility)

(U2, U3): no explicit marker; U2 states a problem, U3 states an action that fixes it -> Problem-Solution

(U3, U4): marker "After that" -> Sequence, and semantically U4 is also the Effect of the restart (Cause-Effect achieved through temporal sequencing)

Step 5 – Discourse structure:

[U1 --Cause-Effect--> U2] --Problem-Solution--> U3 --Sequence/Result--> U4

Explanation

The identified relations make the discourse logically coherent because each unit is explicitly tied to its neighbour by a causal, resolving, or temporal link: the cause explains why the problem occurred, the problem motivates the solution, and the sequence marker signals that the final state follows directly from the solution. This chain of connected relations lets the reader build a single, unified mental model of the events instead of interpreting the sentences as unrelated facts, which is precisely what Rhetorical Structure Theory (RST) uses to define discourse coherence.

Answer 4: Algorithm for Dialogue Act Classification

Problem Understanding

Input: A sequence of conversational utterances between a User and an Agent. Output: A dialogue act label (Greeting, Request, Question, Inform, Offer, Confirmation) for every utterance, based on lexical cues, sentence type, and dialogue context.

Algorithm (Pseudocode)

```
ALGORITHM ClassifyDialogueActs(Utterances U)
1.  prevAct <- NULL
2.  FOR EACH utterance u IN U DO
3.      tokens   <- Tokenize(u)
4.      features <- ExtractFeatures(tokens)       // POS tags, sentence type, cue words, punctuation
5.      IF ContainsGreetingCue(tokens) THEN                // "hello", "hi"
6.          act <- Greeting
7.      ELSE IF IsInterrogative(u) THEN                    // "?" / aux-inversion
8.          act <- Question
9.      ELSE IF ContainsRequestCue(tokens) THEN            // "need", "help", "can you"
10.         act <- Request
11.     ELSE IF ContainsOfferCue(tokens) THEN              // "I can", "I will", "would you like"
12.         act <- Offer
13.     ELSE IF ContainsConfirmationCue(tokens) AND prevAct = Question THEN  //
"sure","okay","yes"
14.         act <- Confirmation
15.     ELSE
16.         act <- Inform                              // declarative statement
17.     dialogueActSequence.append(act)
18.     prevAct <- act
19. RETURN dialogueActSequence
```

Application to the Given Conversation

U1 User : "Hello, I need help with my assignment."

-> contains a request cue ("I need help"); the greeting "Hello" is only the opener, so the dominant communicative intention is asking for help => Request

U2 Agent : "Sure, what problem are you facing?"

-> "Sure" acknowledges, but the utterance is primarily interrogative asking for details

=> Question

U3 User : "I do not understand machine learning."

-> declarative statement giving information about the user's state => Inform

U4 Agent : "I can explain the basic concepts to you."

-> "I can …" is a classic offer-of-service cue => Offer

Dialogue-act sequence: [Request, Question, Inform, Offer]

Explanation

Recognising dialogue acts helps a conversational agent select an appropriate next response because each act predicts the type of response that should logically follow it — a Request is normally answered with an Offer or a clarifying Question, a Question expects an Inform (answer), and an Inform may be followed by an Offer of help or a further Question. By mapping the recognised act to a response-generation policy or state-transition table, the agent can choose the response type that keeps the conversation relevant and on-topic instead of generating a generic or mismatched reply.

Answer 5: Algorithm for Dialogue State Tracking

Problem Understanding

Input: A multi-turn flight-booking dialogue. Output: A continuously updated dialogue state (slot-value pairs for Departure City, Destination City, Travel Date, Number of Passengers) and the next question the agent should ask to fill the remaining missing slots.

Algorithm (Pseudocode)

```
ALGORITHM TrackDialogueState(Utterances U)
1.  state <- { DepartureCity: NULL, DestinationCity: NULL, TravelDate: NULL, Passengers: NULL }
2.  slotPriority <- [DestinationCity, DepartureCity, TravelDate, Passengers]
3.  lastQuestion <- NULL
4.  FOR EACH user utterance u IN U DO
5.      intent   <- DetectIntent(u)              // e.g., book_flight, provide_info
6.      entities <- ExtractEntities(u)           // NER: GPE (city), DATE, NUMBER
7.      FOR EACH entity e IN entities DO
8.          slot <- MapEntityToSlot(e, lastQuestion)  // uses the previous agent question as context
9.          state[slot] <- e.value
10.     missing <- [ s IN state WHERE state[s] = NULL ]
11.     IF missing = EMPTY THEN
12.         nextAction <- ConfirmBooking(state)
```

13. ELSE
14. nextSlot <- HighestPriority(missing, slotPriority)
15. nextAction <- GenerateQuestion(nextSlot)
16. lastQuestion <- nextSlot
17. Agent responds with nextAction
18. RETURN state

Application to the Given Conversation

Turn 1 User: "I want to book a flight."

-> intent = book_flight; no entities extracted; all slots still NULL.

-> Highest-priority missing slot = DestinationCity => Agent asks: "Where would you like to travel?" (matches given dialogue)

Turn 2 User: "I want to go to Delhi."

-> entity "Delhi" (GPE); lastQuestion = DestinationCity => state[DestinationCity] = "Delhi"

-> Remaining missing slots: DepartureCity, TravelDate, Passengers; next priority = DepartureCity

-> Agent asks: "From which city?" (matches given dialogue)

Turn 3 User: "From Chennai."

-> entity "Chennai" (GPE); lastQuestion = DepartureCity => state[DepartureCity] = "Chennai"

-> Remaining missing slots: TravelDate, Passengers; next priority = TravelDate

-> Agent would next ask: "On which date would you like to travel?"

Final dialogue state after the given turns: { DepartureCity: "Chennai", DestinationCity: "Delhi", TravelDate: NULL, Passengers: NULL }

Explanation

Dialogue state tracking improves coherence in a multi-turn conversation because it keeps a persistent memory of exactly which slots have already been filled, so the agent never repeats a question that has already been answered and always asks about the most relevant remaining piece of information next. It also lets short, context-dependent user replies (such as "From Chennai") be correctly interpreted using the slot referenced by the immediately preceding agent question, so the whole exchange stays goal-directed and contextually consistent from start to finish.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	3	3	3	3	3	15	75
2	3	3	3	3	3	15	
3	3	3	3	3	3	15	
4	3	3	3	3	3	15	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____